PROJECT-BASED LEARNING
PHASE-I EVALUATION

Class Pilot

your smart class attendance assistant

Team ID	DSCPP-III-2026-T340
Domain	Data Structures & C++
Department	Computer Science & Engineering
Institution	Graphic Era Deemed To Be University , Dehradun
Academic Session	2026-27

Team

Siddhant Gupta – 2510011681 | Sec: C

Abheer Sharma – 2510011271 | Sec: G

Simran Bhagat – 2510012722 | Sec: K

Radhika Aggarwal – 2510011743 | Sec: H

Mentor

Mr. Kireet Joshi

1. Project Overview

Class Pilot is a C++ application made for attendance planning. We noticed that just knowing the current attendance percentage is not always enough. A student may also want to know what will happen if a few upcoming classes are missed. The application uses the old attendance record and the coming timetable to give that kind of estimate.

The idea can also work in places other than a college. For example, it can be useful in schools, coaching classes, certification courses, vocational training, workshops, internships and employee training. The common point in all these cases is that participation has to stay above some required level.

2. Real-World Problem

The main problem is not simply recording whether someone was present. The useful question is what happens to the required participation percentage when a person decides to miss a particular session.

A person can miss a class for many normal reasons. Travel, family work, an appointment, an interview, job duties or a timetable clash are a few examples. Doing the percentage calculation again and again is also inconvenient. It becomes easier to make a mistake when the timetable, holidays or required percentage changes.

For this reason, Class Pilot considers both the attendance recorded so far and the sessions that are coming next. This helps the user understand how many upcoming sessions they can miss, how much attendance may be needed later, and what percentage they are likely to have after a planned absence.

- A learner can check the possible effect of missing a session before deciding, instead of estimating it mentally.
- Teachers and coordinators can avoid repeating the same attendance calculations for different records.
- A person in a training or certification programme can compare the attendance requirement with other commitments such as work or travel.
- The system can also make participation information easier for an organisation to understand. Class Pilot is not intended to force anyone to attend. Its purpose is to show the information clearly so the user can make the decision with a better idea of the result.

4. Main Working

1	Enter the course/activity, timetable, required percentage and previous attendance.
2	Mark each completed session as present or absent.
3	Prepare the upcoming sessions from the timetable and working dates.
4	Check a possible plan, such as missing some future sessions.
5	Show the expected percentage and the records that need attention.

5. Where We Use C++ and DSA

The prototype is written in C++. We use different data structures based on the tasks they handle. They help store records, find activities, manage future sessions and rank risky attendance records. C++ builds the application, while DSA manages its data and operations.

Attendance records	Classes/structs with vector	Dynamic array / traversal
Activity lookup	unordered_map	Hash table
Upcoming sessions	queue	Queue
Risk ranking	priority_queue	Priority queue / heap
Ordering results	std::sort + comparator	Sorting
Finding records	STL operations / loops	Searching
Program structure	Functions, OOP, validation	Algorithm design + complexity

6. Database: Where Our Data Will Be Stored

We plan to store the prototype data in a local SQLite database. Since SQLite works through a database file, we do not need a separate database server. The database can simply be kept as attendance.db inside the project's data folder.

When the user adds attendance or changes a schedule the C++ program reads the required records. Saves the changes in SQLite.

Storage plan: C++ application → SQLite → attendance.db containing Users, Activities, Sessions, Attendance and Settings.

7. DSA Algorithm we use

We use a priority queue for risk ranking in the DSA part.

The program first calculates the participation percentage and checks how much room is left before the requirement becomes a problem.

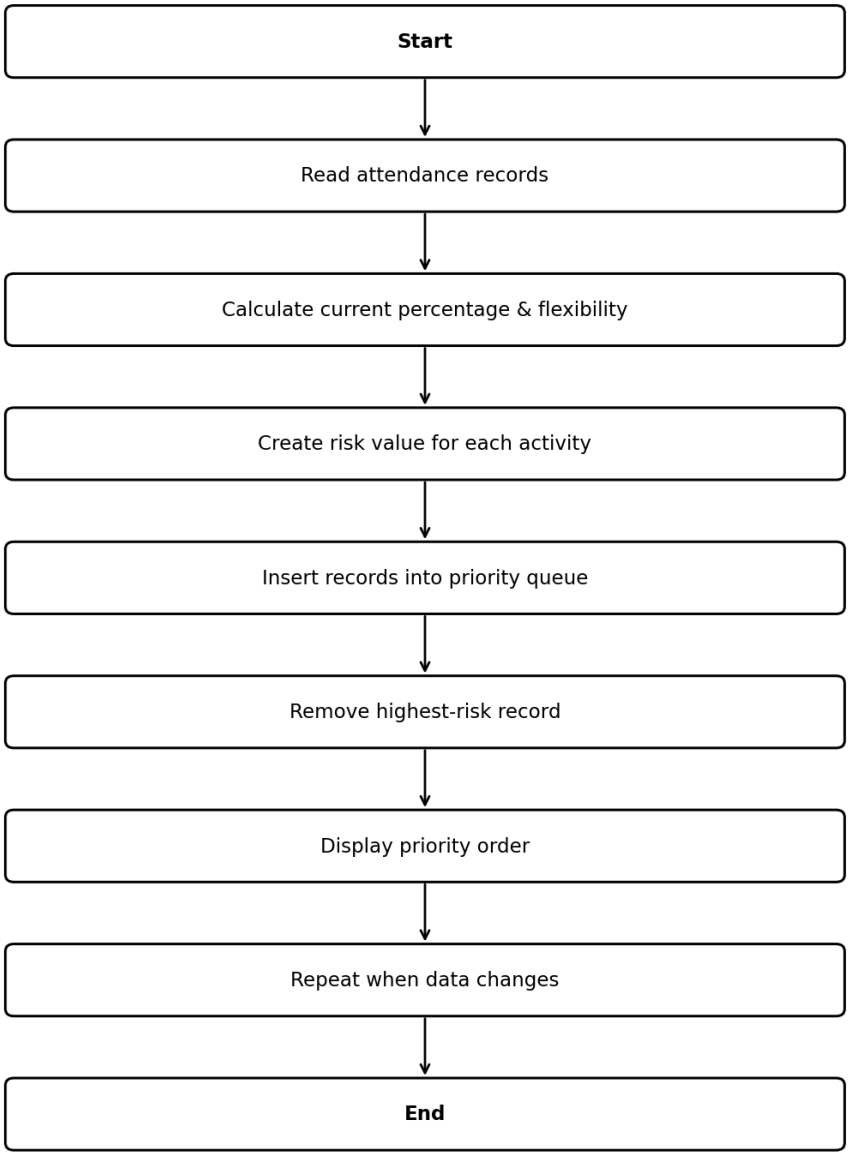
A risk value is then given to each activity.

The priority queue keeps the higher-risk records at the top so they can be shown first.

The priority queue has an use in the project.

The priority queue helps the application decide which attendance record needs attention of being present only, for demonstrat

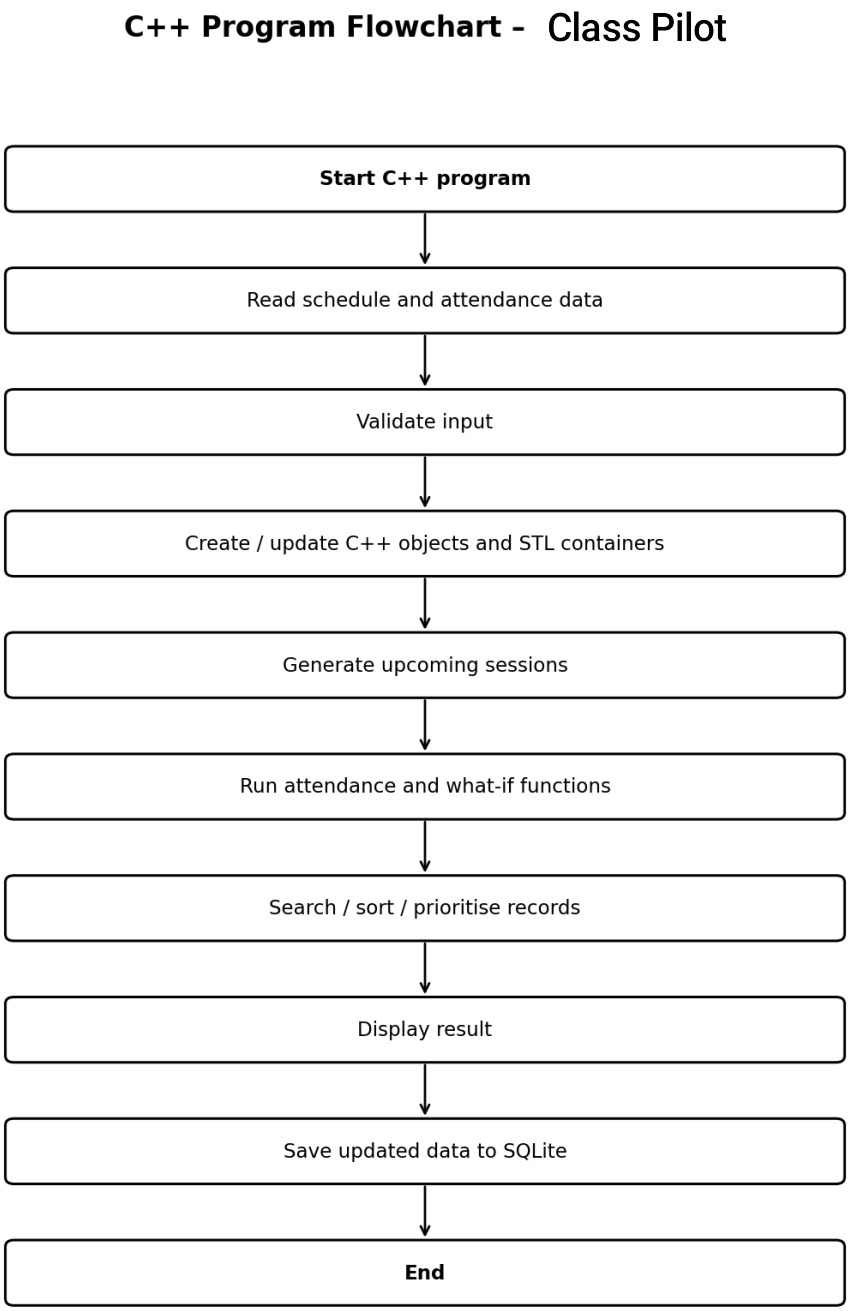
DSA Flowchart – Risk Ranking



8. C++ Algorithm / Program Logic

The C++ program logic is the overall sequence followed by the application. It takes the input, checks that input, works on the attendance records, performs the required calculations, updates the SQLite data and shows the result on the screen.

9. C++ Flowchart



10. Placement-Relevant C++ & DSA

The project uses several C++ and DSA topics that can also be useful in coding tests and interviews. These include OOP, STL, vectors, strings, hashing, queues, priority queues, sorting, searching, comparators and time complexity.

OOP: Activity, Session and Attendance can be represented as separate classes.

- STL: vector, unordered_map, queue, priority_queue and sort are used for different parts of the program.
- Complexity: a hash table gives average O(1) lookup, while normal comparison-based sorting generally takes O(n log n).
- Problem solving: the attendance problem is converted into suitable data structures, algorithms and functions.
- Interview discussion: the team can explain the reason for using a queue, hash table or priority queue for a particular operation.

DSA learning can also support preparation for technical interviews. A SIGCSE study discussed interview-focused activities and how they can help students understand recruitment and prepare for interviews.

11. Technical Feasibility

Hardware	A regular laptop or desktop is enough for the prototype.
Software	The prototype needs a C++17-compatible compiler and the SQLite library.
Database	SQLite keeps the prototype records in one lightweight local database file.
Performance	The selected vectors, hashing, queues and priority queues are sufficient for the expected data.
Development	Breaking the project into smaller modules makes it practical to complete during a semester.
Cost	The prototype can be made with commonly available and free development tools.

The project is suitable for a semester-level prototype. A normal laptop or desktop is enough for the application, and SQLite can handle the local records. The selected data structures are also enough for the amount of data expected in this project. Class Pilot is a planning and decision-support prototype and is not meant to replace an organisation's official attendance system.

12. Research Support

For the project, we referred to published work related to digital attendance systems and DSA-based interview preparation. These papers give background for the problem and for some of our technical choices. They should not be taken as saying that the exact Class Pilot system is already available in those

- **Implementation and Evaluation of Technical Interview Preparation Activities in a Data Structures and Algorithms Course — Kapoor, Panchal & Gardner-McCune, ACM SIGCSE, 2023. This study looks at using technical-interview preparation activities along with DSA learning. DOI: 10.1145/3545945.3569755**
- **An Empirical Study of Programming Languages Specified in Engineering Job Postings — Schibelius, Ross & Katz, ASEE Annual Conference, 2022. This paper studies programming languages mentioned in engineering job advertisements and includes C++ among the languages found often in its dataset. DOI: 10.18260/1-2--41235**

13. Example of the System

As an example imagine a participant in a training programme. That participant has attended thirty-four of forty sessions. The minimum required participation is seventy-five percent. Therefore the present percentage, for that participant is eighty-five percent. The participant now has another commitment and is considering missing the next two sessions.

Class Pilot can use the upcoming sessions to test this case. The two planned absences are included in the calculation and a new percentage is obtained. If it stays at or above 75%, the user can see that result. If it falls below the requirement, the system can show the attendance that will be needed to recover.

14. Advantages and Limitations

- Simple: The user can get the result they need without doing the calculation each time.
- Broader use: the same type of planning can be used beyond college attendance.
- Practical DSA: the structures used in the project are linked to functions of the application.
- Placement value: the project gives the team examples of C++ OOP and STL that can be explained in interviews.
- Limitation: the prediction depends on the attendance. Timetable information entered by the user.
- Limitation: the present prototype cannot automatically take attendance records from an external organisation.

15. Final Summary

Class Pilot is designed around an everyday situation.

A student may need to miss a class but may be unsure, about how it will affect their attendance.

Of showing only the current percentage the application also considers the upcoming timetable.

This allows the user to check the outcome before deciding to miss a session.

The application uses C++ for its functionality and SQLite to store local records.

Hashing, queues, priority queues, searching and sorting are used to process and organise th